Universität Konstanz

Introduction to Computing for the Social Sciences

Session 1 – Introduction

David Garcia

Winter Semester 2026-2027

About me

Universität Konstanz

- B.Sc. Computer Science – Universidad Autónoma de Madrid
- M.Sc. Computer Science – ETH Zurich
  - Theory of computation track
- Doctorate at ETH Zurich: Department of Management, Technology and Economics
- Habilitation at ETH Zurich – Now Privatdozent teaching Social Data Science
- Group leader at the Medical University of Vienna
- Faculty member at the Complexity Science Hub Vienna
- Professor for Computational Behavioral and Social Sciences
  Graz University of Technology
- Since 1.10.2022:
  Professor for Social and Behavioral Data Science – University of Konstanz
  Leading the Social Data Science lab

The Social Data Science Lab


David Garcia


Giordano de Marzo


Alessandro Bellina

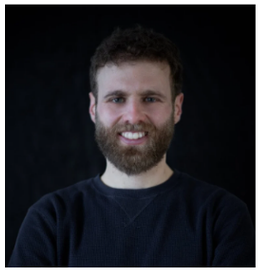

Taehee Kim

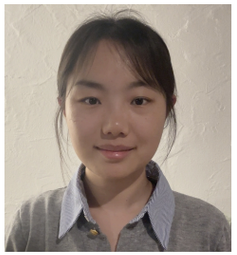

Farah Adeeba

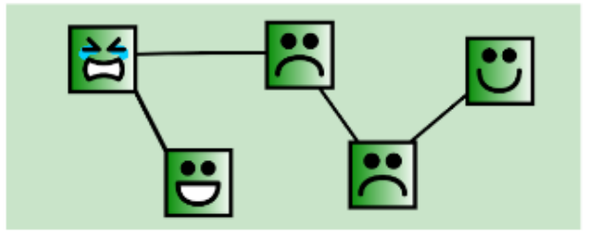

Anna Di Natale


Peer Saleth


Elena Solar


Stephen Tyndel

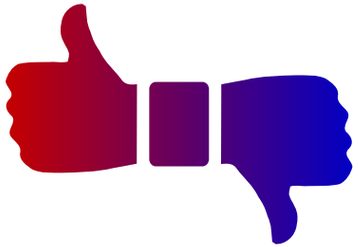

Jing Ma



Computational Affective Science

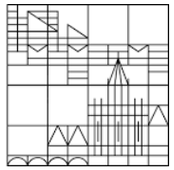


Information Disorder



Polarization and Inequality

Universität Konstanz



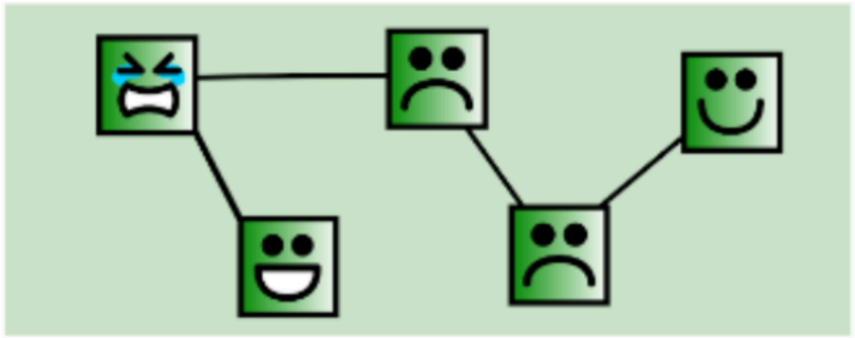


Complexity Science Hub

Our Research Topics



Methods in CSS



Computational Affective Science



Information Disorder

The ICSS Team

Universität Konstanz

Lecturer: Prof. Dr. David Garcia
Consultation: book a consultation hour by email, I come to the university every week

Lecturers of Practice part: Elena Solar, David Garcia

Tutors for Practice part: Aaron Siebert, Eli Schwarer, Anna Luise Harnischmacher

Lecture: Wednesdays 10:00-11:30, D301

Exercises: Tuesdays 15:15-16:45, D301

Outline

1. Course Motivations and Objectives
2. Course Overview
3. Course Formalities
4. A Brief Introduction to Computer Science

The Digital Society

Universität Konstanz

More and more of our actions happen through digital technologies, e.g.:

- Searching for a job online
- Talking with a family member over the phone
- Sharing open source code
- Posting a video on a social networking site
- Paying at the supermarket with a credit card
- Chatting with your boss on a company instant messenger



Motivation I: The Digital Transformation

Universität Konstanz

The growth in adoption and evolution of these technologies is the **digital transformation** of society, which has some consequences:

- **Datafication:** Digital technologies generate data traces that can persist and be analyzed
- How individuals interact with digital technologies is based on the code of software running on them. The design of that code is based on algorithms that describe the rules of how the technology operates
- Digital technologies might use data produce by humans in their algorithms, for example through machine learning models to personalize content or make predictions

Digital trace data



Facebook: Visualizing Friendships

Motivation II: the Computational revolution

Universität Konstanz

- Computational methods to handle and analyze new social data
  - automation and scalability: we cannot process everything by hand
  - robustness: analysis changing assumptions or parameters
  - reproducibility: code for anyone to build and replicate
- BuT lack of fundamental understanding of the “tools” we use
  - often only a working knowledge of high-level software programs
    e.g. Excel, SPSS, STATA
  - complex research problems require more **advanced computational skills**
    - automate data retrieval and handling
    - design efficient algorithms and run them in computational infrastructure
    - develop software that implement specific methods or solutions

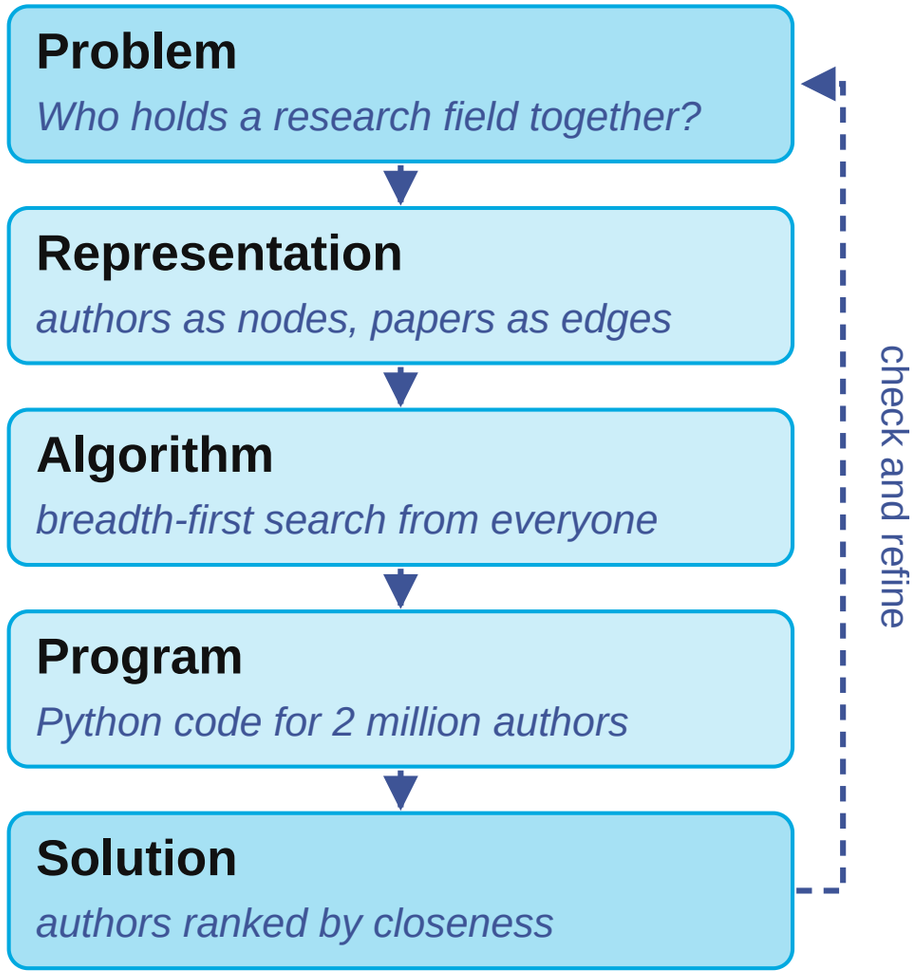
Motivation III: Challenges in Data Science

Understanding computation is a requirement to understand current issues in the digital society, technology, and social science research

- **Platform biases:** The design of platforms and their algorithms can affect the behavior we observe. Some platforms have non-human actors (e.g. bots) that affect the data.
- **Data gatekeepers:** Not everyone can access the same data. Some research might be impossible to reproduce.
- **Performative behavior:** Talking online is not the same than offline. We have to consider what is the purpose of the people involved and if that distorts their behavior.
- **Representativity issues:** Not everyone leaves digital traces and measurement methods or predictive models can have biases that lead to discriminatory outcomes.
- **Privacy and ethics:** data is people – privacy expectations and downstream consequences of technology are new ethical issues.
- **The data deluge:** Too much data enables black-box predictions that can be useful but limit our understanding.

Goals of the Course

- **Overarching goal: being able to solve problems computationally**
- Appreciate, understand and use ideas at the core of computer science and programming
- Demonstrate familiarity with major algorithms and data structures
- Use computational techniques for solving problems in the social sciences
- Have familiarity with Python programming language for simple computational tasks



Example of computational solution

Course Overview

Universität Konstanz

1. Course Motivations and Objectives
2. **Course Overview**
3. Course Formalities
4. A Brief Introduction to Computer Science

Overview of the Course

- Fundamental principles of Computer Science & basic programming concepts
  - Topics covered:
    - Principles of information coding
    - Computer systems & information storage
    - Data types, data structures & algorithms
    - Programming paradigms and basic software engineering
    - Basic database and parallel programming concepts
  - Format:
    - Concepts and theory are taught in the lectures
    - **Pen-and-paper exercises apply concepts: Knowing is doing**
    - Link to practical part, course calendars algined

Lectures Outline

- Session 1: Introduction
- Session 2: Information Coding
- Session 3: Data Structures
- Session 4: Software
- Session 5: Algorithms
- Session 6: Recursion
- Session 7: Sorting Algorithms
- Session 8: Algorithmic Complexity
- Session 9: Theoretical Computer Science
- Session 10: Databases
- Session 11: Machine Learning
- Session 12: Parallel Programming
- Session 13: Social Science Applications
- Session 14: Exam Review

Course calendar

- **01** Introduction (21.10 – 27.10)
- **02** Information coding (28.10 – 03.11)
- **03** Data Structures (4.11 – 13.11)
- *No class on 10.11 nor 11.11.*
  Instead on 13.11 in A702:
  Exercise 03 at 10:00-11:30,
  Lecture 04 at 13:30-15:00
- **04** Software (13.11 – 17.11)
- **05** Algorithms (18.11 – 24.11)
- **06** Recursive algorithms (25.11 – 01.12)
- **07** Sorting Algorithms (02.12 – 08.12)
- **08** Algorithmic Complexity (09.12 – 15.12)
- *No class on 22.12 nor 23.12*
- **09** Theoretical Computer Science (16.12 - 12.01)
- **10** Databases (13.01 - 19.01)
- **11** Machine Learning (20.01 - 26.01)
- **12** Parallel Programming (27.01 – 02.02)
- **13** Social Science Applications (03.02)

Course Formalities

1. Course Motivations and Objectives
2. Course Overview
3. **Course Formalities**
4. A Brief Introduction to Computer Science

Structure

Universität Konstanz

- Theory course: lectures + exercises
  - 90 min lecture (David): Wednesdays 10:00 – 11:30 in D301
  - 90 min exercise session: Tuesdays 15:15 – 16:45 in D301
    - Pen-and-paper exercises linked to the lecture content
    - Only useful if you work on them in advance
- Practice course: tutorials and the four programming assignments
  - Separate course with its own schedule, see its syllabus

ICCS (Theory) Grading

Universität Konstanz

Introduction to Computing for the Social Sciences (Theory)
POL-31760 (6 ECTS)

- Final 90-minute written exam on concepts and foundations explained in lectures
- Pen-and-paper exercises are examples of exam questions
- First date in February. Resit in March. You cannot go to the resit if you passed the first
- Grade completely independent from ICSS (Practice)
- You will have to register for both exams (and the Python block course exam) in Dec/Jan.

Final written exam

Universität Konstanz

- The final exam will take 90 minutes and its grade will be the final grade of the theory part of the course
- Closed book, no calculators allowed
- Pen-and-paper exercises are examples of written exam exercises. Do try them before the solutions session and come to show your solutions.
- Early February: extra Q&A and exam example session
- Learning in Computer Science means generalizing. To learn more, do other exercises, check reading materials, and ask for more in our sessions.
  - For example, you have to be able to approach a computational problem we have never seen in class and to understand the properties of the solution.

Materials: Github and Ilias

Universität Konstanz

- You will find all course materials on Github:

https://github.com/dgarcia-eu/ICSS

- Includes Lecture slides, exercise sheets, and solutions
- New this year: AI-assisted transcripts. All content is approved by the lecturer
  - Extra appendix in transcripts with interesting context on the topic
  - Feedback on quality, improvements, or errors is welcome
- Read the syllabus carefully. It includes a calendar and grading information
- Join the course Ilias to stay up to date:

https://ilias.uni-konstanz.de/goto.php/crs/2103977

- Email announcements are sent to students enrolled in Ilias

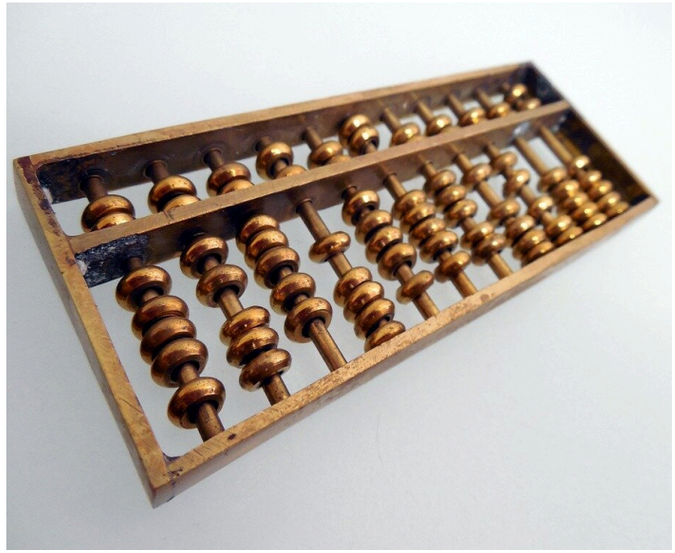
A Brief Introduction to Computer Science

Universität Konstanz

1. Course Motivations and Objectives
2. Course Overview
3. Course Formalities
4. **A Brief Introduction to Computer Science**

What is a Computer?

- What is a computer?
- “A machine that stores and manipulates information under the control of a changeable program” (Zelle, 2016)



stores and manipulates information but not a machine (not automated)



automatically stores and manipulates information but not programmable



Today: Personal computers, servers, smartphones, fridges, photo cameras, watches, etc

What is Computer Science?

Universität Konstanz

- What is computer science?
  - scientific and practical approach to computation
  - enable collection, representation, processing, storage and access to information
  - study of theory of computation, algorithmic processes, and design of computational machines
  - “Computer science is no more about computers than astronomy is about telescopes.”
    (Attributed to Edsger W. Dijkstra)
- Questions in computer science
  - What can be computed? (Finding what is computable)
  - How can we solve a problem with computation?
    (Designing algorithms, i.e. recipes to solve problems)

What is an algorithm?

An algorithm is a self-contained step-by-step set of operations to solve a given problem. Algorithms perform calculation, data processing, and/or automated reasoning tasks.

- Example: SelectionSort
- Problem: Sort a list of elements
- The algorithm is the set of step-by-step operations to solve the problem
- Idea: find the smallest and swap it with the first one not sorted yet

Procedure SELECTION-SORT(*A*, *n*)
Inputs:
A: an array
n: the number of elements in *A* to sort
Result: the elements of *A* are sorted into nondecreasing order
For *i* = 1 to *n* - 1:
    smallest ← *i*
    For *j* = *i* + 1 to *n*:
        If *A*[*j*] < *A*[*smallest*]
            then *smallest* ← *j*
    swap *A*[*i*] with *A*[*smallest*]

What is an algorithm?

Universität Konstanz

Python Code:

```python
def selection_sort(A, n):
    for i in range(n-1):
        smallest = i
        for j in range(i+1, n):
            if A[j] < A[smallest]:
                smallest = j
        # Swap
        temp = A[i]
        A[i] = A[smallest]
        A[smallest] = temp
    return A
```

Pseudocode:

Procedure SELECTION-SORT(*A*, *n*)

Inputs:

A: an array

n: the number of elements in *A* to sort

Result: the elements of *A* are sorted into nondecreasing order

For *i* = 1 to *n* - 1:

    smallest ← *i*

    For *j* = *i* + 1 to *n*:

        If *A*[*j*] < *A*[*smallest*]

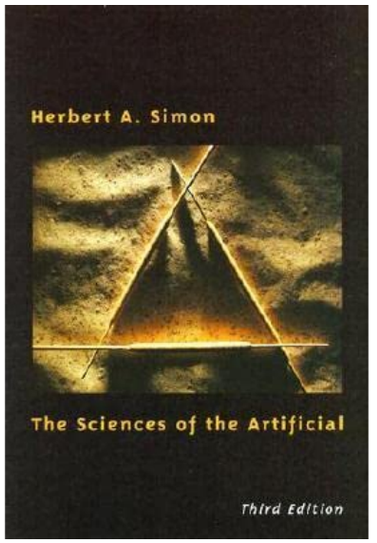
            then *smallest* ← *j*

    swap *A*[*i*] with *A*[*smallest*]

The Sciences of the Artificial

- Herbert Simon (1916 – 2001): Political scientist with great influence in Economics (Nobel Prize) and Computer Science
- Artificial Sciences versus Natural Sciences
  - Artificial sciences study what humans create
  - Artificial sciences are not only *descriptive* but also ***prescriptive***: they try to find what we should do, not just how things are
  - E.g. economists look for a tax incentive to drive innovation
  - E.g. sociologists look for a policy to reduce gender inequality
  - E.g. computer scientists look for programs that guide a robot through a maze, or decide whom to interview for a job, who gets parole, or where to place refugees
- Prescriptive sciences study phenomena with a *purpose*





Antecessors of Computer Science

~ 300 B.C. **Euclidian algorithm**

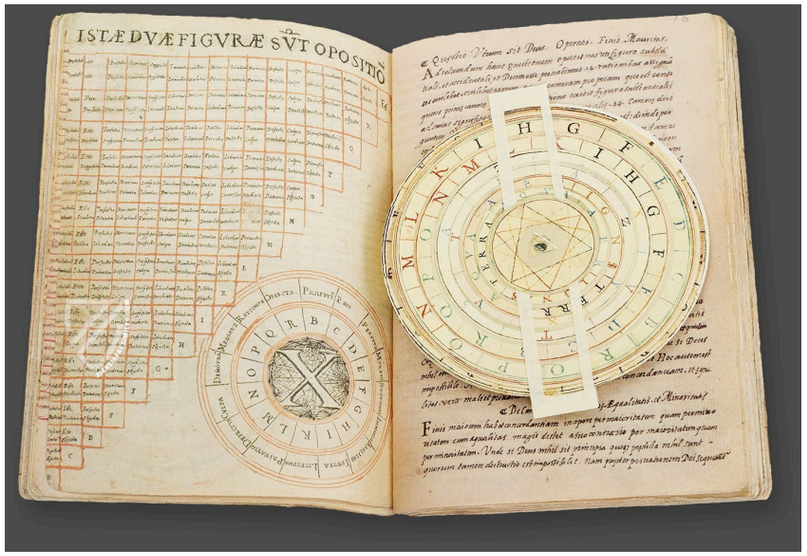
820 A.D. **Al-Khwarizmi lays foundation for “algorithms”**

1305 **Ramon Llull’s mechanical logic**





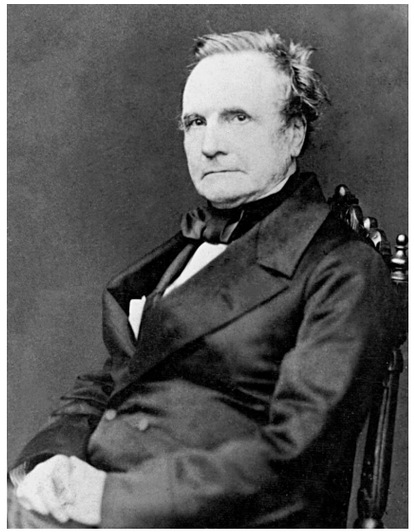




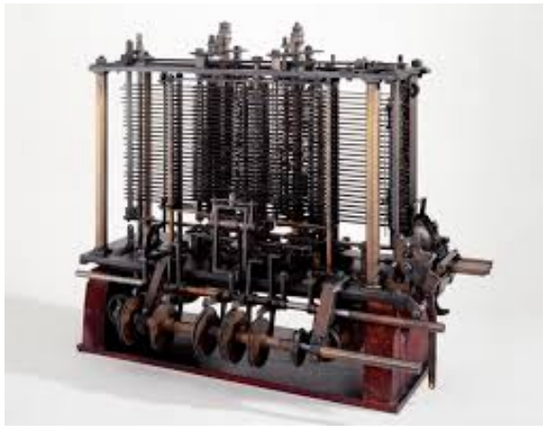
The First Concept of Computers

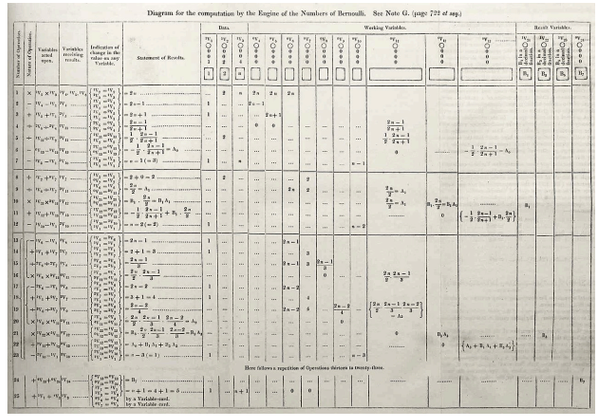
1837 **Charles Babbage's Analytical Engine**

1843 **Ada Lovelace's first computer program**





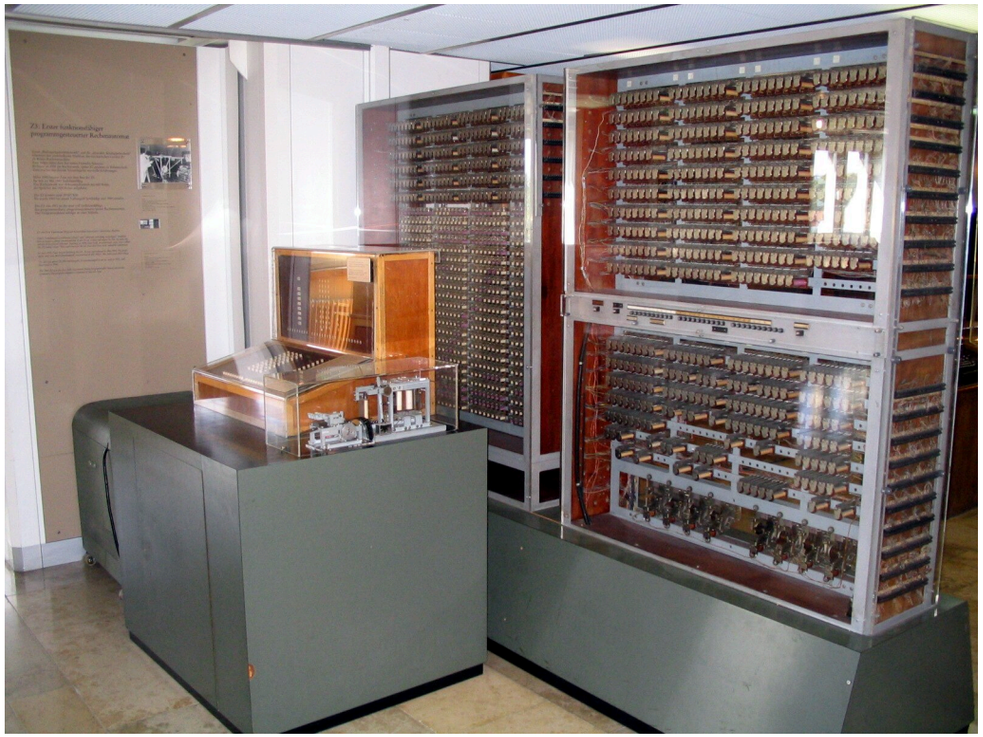


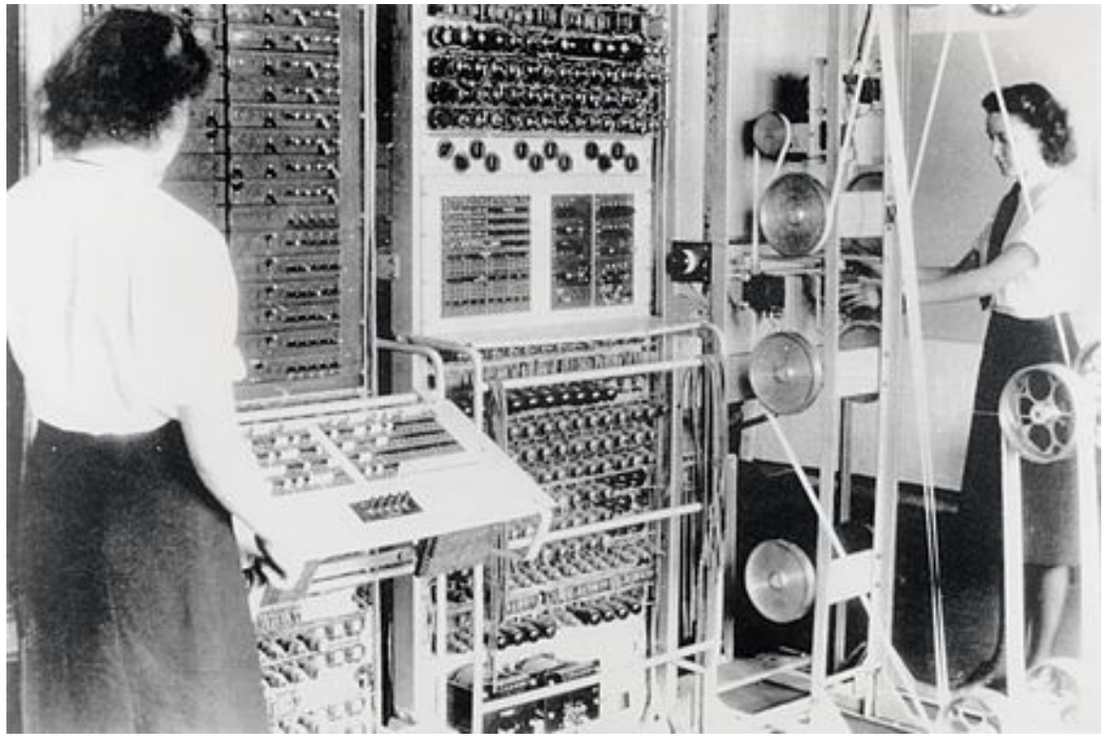


The First Real Computers

1941 **Konrad Zuse's first functioning computer Z3**

1944 **Colossus Mark 2 computer breaks the Lorenz cipher**

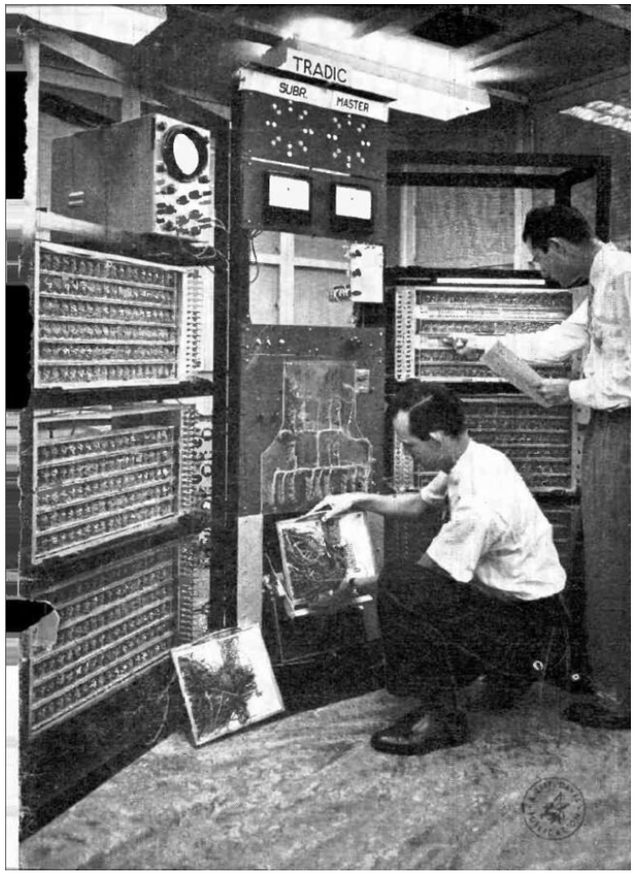


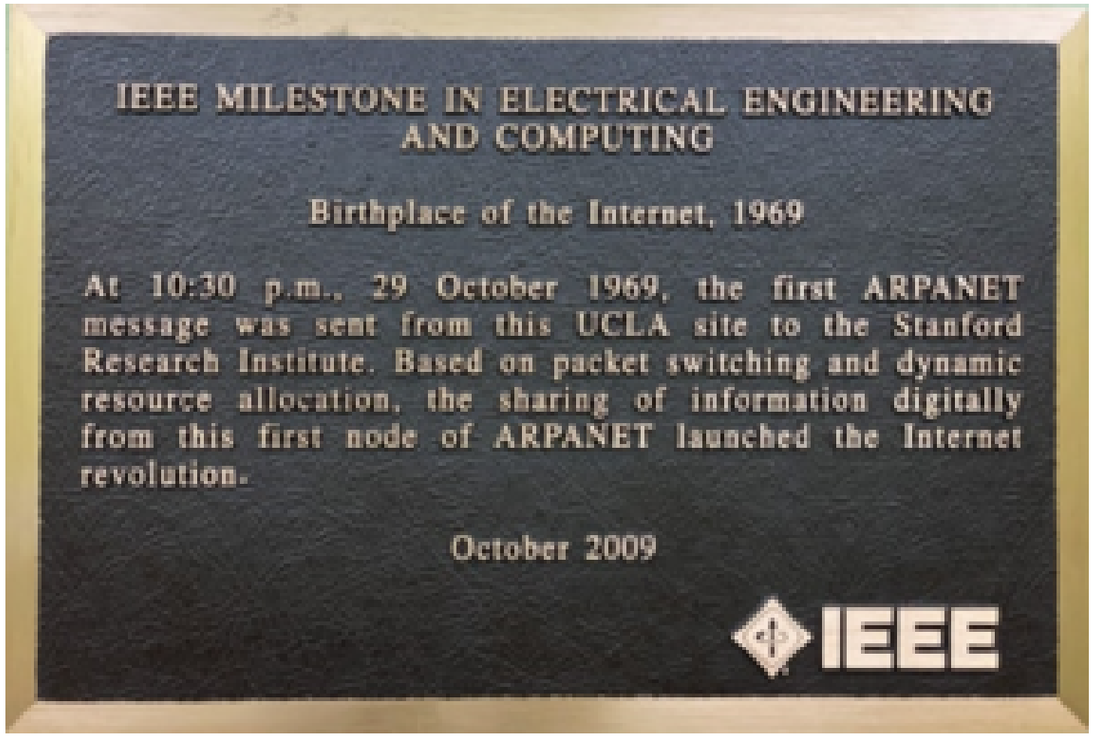


Computing Goes Industrial

1954 **First transistor computer TRADIC at Bell Labs**

1969 **First message sent via the *ARPANET***



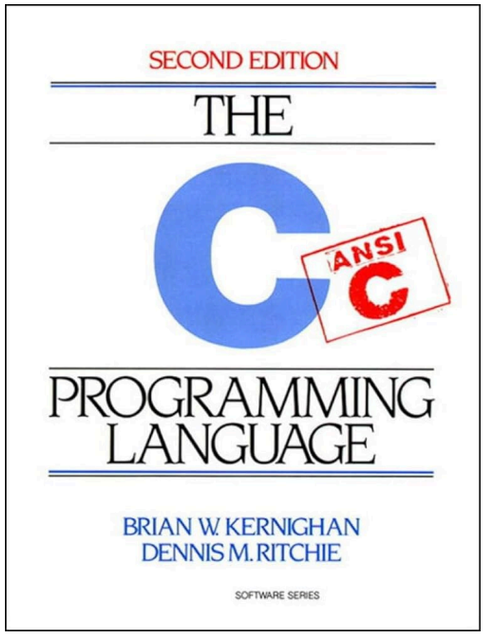


The PC era

1972 **Introduction of the *C* programming language**

1981 **IBM PC delivered**

1990 - **Development of the WWW at CERN, Switzerland**







The AI era

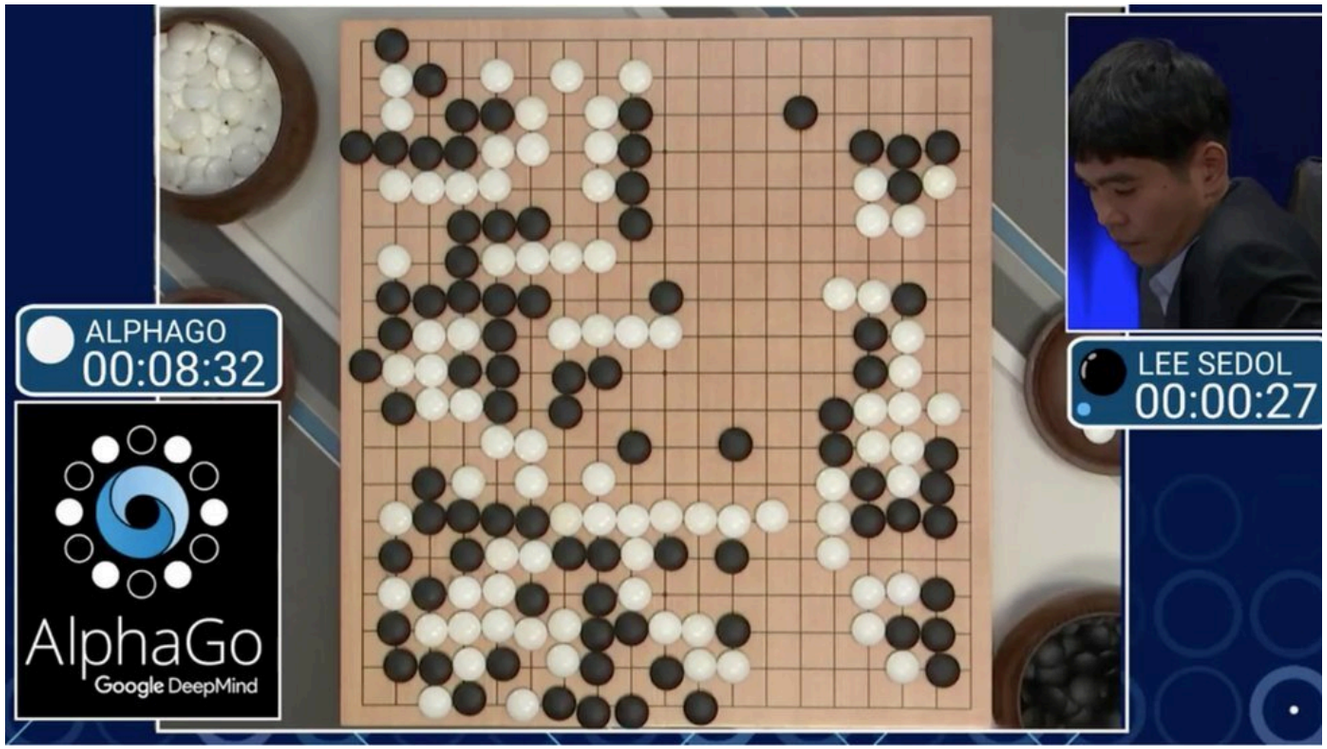


1997 **Deep Blue beats Garry Kasparov**

2016 **AlphaGo beats Lee Sedol**

2022 **AI becomes popular with ChatGPT**

ChatGPT

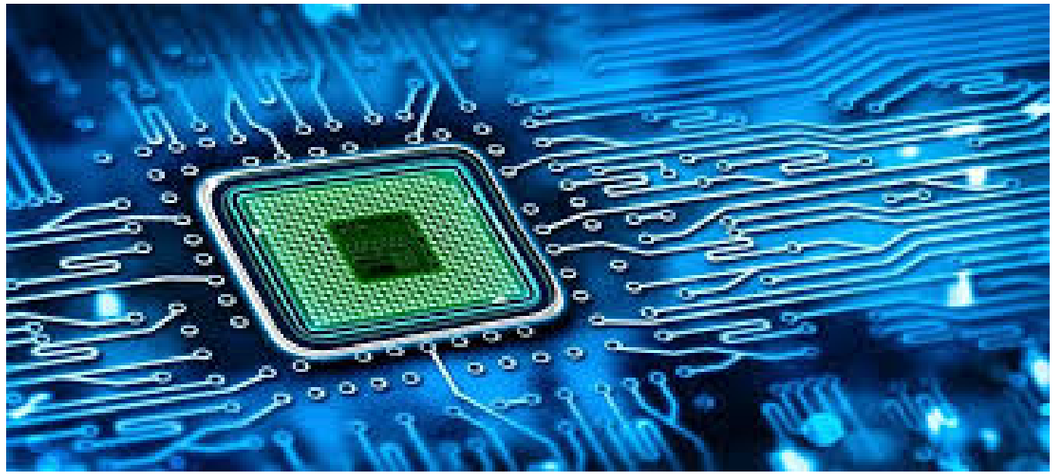


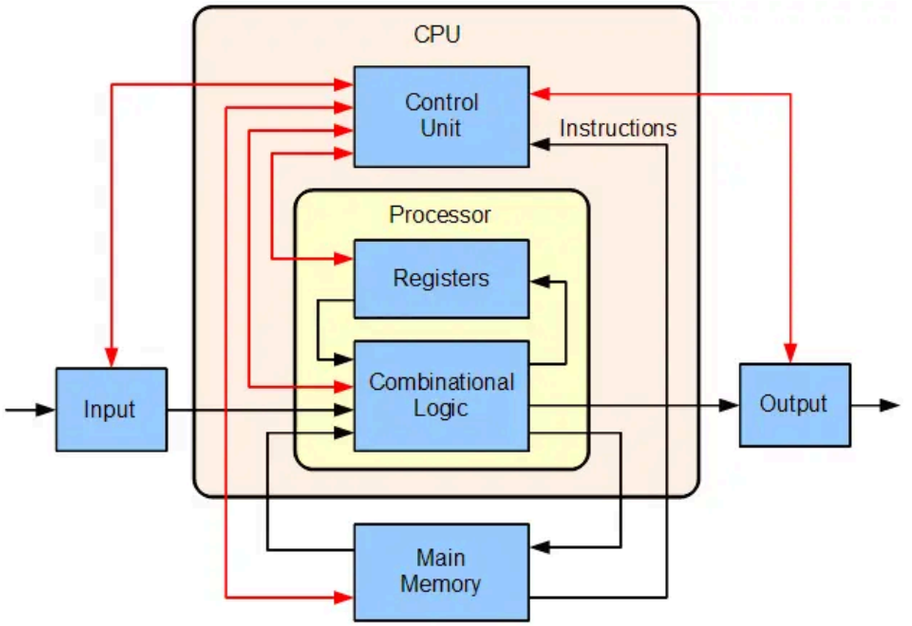
Subfields of Computer Science

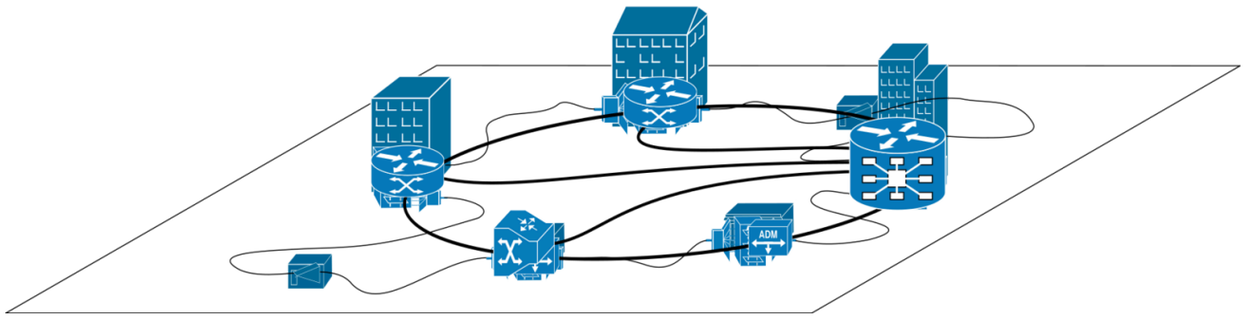
Universität Konstanz

Computer Engineering

- Microprocessor technology
- Computer architecture
- Communication between machines



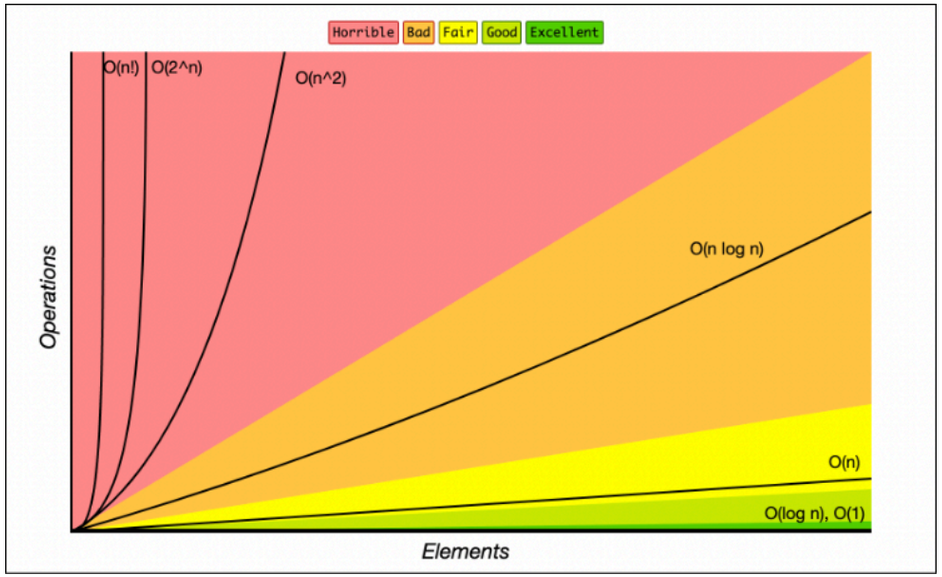


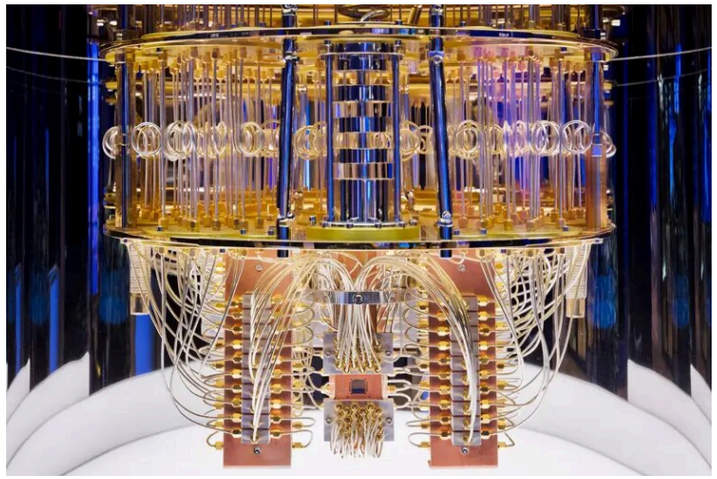


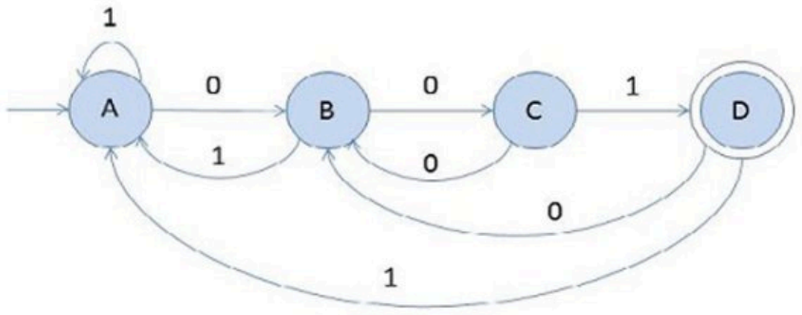
Subfields of Computer Science

Theoretical Computer Science

- Automata theory and formal languages
- Complexity theory
- Quantum computing







Subfields of Computer Science

Applied Computer Science

- Programming languages
- Algorithms and data structures
- Operating systems
- Databases
- Software development



Queue Data Structure



Subfields of Computer Science

Universität Konstanz

Interdisciplinary Fields

- Business informatics
- Computer vision
- Artificial intelligence
- Computational linguistics
- Bioinformatics
- Medical informatics
- Media informatics
- Human-Computer Interaction (HCI)
- **Computational Social Science**





Literature

- No single book covers the course; each lecture gives references for its topic
- Two broader reference books complement the content taught in the lectures
  - ‘Introduction to Computing - Explorations in Language, Logic, and Machines’ by Evans
  - Algorithms Unlocked by Thomas H. Cormen
  - ‘Introduction to Computation and Programming Using Python’ by Guttag
- For advanced reading on algorithm and data structures:
  ‘Introduction to Algorithms’ by Cormen, Leiserson, Rivest, and Stein
- Aim of new transcripts: To provide a self-contained alternative to jumping around book chapters and to harmonize terminology and nomenclature over the whole course

Summary

- Computing is now part of how social science collects, measures and models data
- A computer is a machine that executes a stored program on data, following instructions exactly
- Computer science studies what can be computed, and how efficiently
- An algorithm is a self-contained, step-by-step set of operations; selection sort was our first example
- Computing is a science of the artificial, with a history from the abacus to AI
- The course pairs theory (lectures and exercises) with practice (Python tutorials)

Next session: Information Coding